

Datenbanken

Vorlesungsskript für das 3. Semester

Studiengang Internationale Medieninformatik

11. Datenbankskripte am Beispiel von PL/pgSQL – Teil 2

Dozent: M. Sc. Burak Boyaci

Version: 13.01.2026

Wintersemester 25/26

Dieses Skript unterliegt der *Creative Commons License* CC BY 4.0
(<http://creativecommons.org/licenses/by/4.0/deed.de>)



14

PL/pgSQL – Datenbankskripte Teil 2

Kapitelübersicht

14.1 Funktionen in PL/pgSQL.....	1
14.2 Prozeduren in PL/pgSQL	4
14.3 Trigger in PL/pgSQL.....	6

14.1 Funktionen in PL/pgSQL

Eine Funktion ist in PL/SQL eine Möglichkeit - ähnlich wie andere Programmiersprachen - bestimmte sich wiederholende Abfragen in einer aufrufbaren Form zu speichern. Funktionen haben den Block-Aufbau wie im letzten Kapitel beschrieben. Eine Funktion gibt immer ein Ergebnis zurück, das über *RETURN* ausgegeben wird.

Beim Erstellen einer Funktion oder Prozedur können Parameter definiert werden. Es gibt drei Arten von Parametern, die deklariert werden können:

IN - Der Parameter kann von der Prozedur oder Funktion referenziert werden. Der Wert des Parameters kann von der Prozedur oder Funktion nicht überschrieben werden.
(= Default)

OUT - Der Parameter kann nicht von der Prozedur oder Funktion referenziert werden, der Wert des Parameters kann jedoch von der Prozedur oder Funktion überschrieben werden.

IN OUT - Der Parameter kann von der Prozedur oder Funktion referenziert werden, und der Wert des Parameters kann von der Prozedur oder Funktion überschrieben werden.

Der Aufbau einer Funktion entspricht dem Block von PL/pgSQL. Die Syntax ist folgendermaßen (eckige Klammern enthalten optionale Bestandteile der Funktion).

CREATE [OR REPLACE] FUNCTION

Name_der_Funktion [(Parameter_der_Funktion)]

RETURNS Datentyp **\$\$**

[DECLARE]

BEGIN

Execution;

RETURN;

[EXCEPTION];

END;

\$\$

LANGUAGE

PLPGSQL;

Funktionen geben immer einen Wert zurück, der für weitere Berechnungen genutzt werden kann. Damit sind Funktionen in PL/pgSQL eine Möglichkeit, sich oft wiederholenden Code einfach und schnell auch öfters auszuführen.

Beispiel:

In unserem Datenmodell *classicmodels* existiert die Tabelle *Customers*, die Kundendaten verschiedener Kunden beinhaltet. Den Primärschlüssel der Tabelle bildet die Spalte *Customernumber*. Wir möchten im Schema *classicmodels* eine Funktion *avg_creditlimit* schreiben, die das durchschnittliche Kreditlimit aller Kunden bestimmt und es an die Variable *v_creditlimit* übergibt. Auf der Konsole soll die Information 'Das mittlere Kreditlimit beträgt: ' und das durchschnittliche Kreditlimit aller Kunden angezeigt werden.

```
create or replace function classicmodels.avg_creditlimit ()
returns decimal
as $$
declare
v_creditlimit
classicmodels.customers.creditlimit%type; begin
select avg(creditlimit) into v_creditlimit from
classicmodels.customers; raise info 'Das mittlere Kreditlimit beträgt:
%', v_creditlimit;
return v_creditlimit;
end;
$$ language plpgsql;
```

Aufruf einer Funktion

Eine Funktion wird in PL/pgSQL über ein SELECT Statement aufgerufen. Wir möchten das durchschnittliche Kreditlimit aus der Tabelle *customers* durch Aufruf der Funktion ausgeben

→ `select * from classicmodels.avg_creditlimit`

() Ergebnis:

Query	Query History	Data Output	Messages	Notifications
INFO: Das mittlere Kreditlimit beträgt: 69167.226890756303				
Successfully run. Total query runtime: 117 msec. 1 rows affected.				

Beispiel 1 mit OUT Parameter:

Eine Funktion wird in PL/pgSQL über ein SELECT Statement aufgerufen. Wir möchten die Kundendaten des Kunden mit der Kundennummer 124 ausgeben:

```
create or replace function classicmodels.get_customer_data (  
  IN v_customernumber int,  
  OUT v_customername text  
)  
as $$  
begin  
  select customername  
  into v_customername  
  from classicmodels.customers  
  where customernumber = v_customernumber;  
end;  
$$ language plpgsql;
```

```
select * from classicmodels.get_customer_data (124)
```

Ergebnis:

	get_customer_data text	🔒
1	Mini Gifts Distributors Ltd.	

Beispiel 2 mit OUT Parameter:

Wir möchten jetzt mehrere Daten des Kunden mit der Kundennummer 103 ausgeben:

```
create or replace function  
classicmodels.get_customer_data3 (  
  out v_customernumber int,  
  out v_creditlimit int)  
as  
$$  
begin  
  select min(customers.customernumber), max(customers.creditlimit)  
  into v_customernumber, v_creditlimit from classicmodels.customers;  
end;  
$$ language plpgsql;
```

```
select * from classicmodels.get_customer_data3 ()
```

Ergebnis:

Data Output	Messages	Notifications
       		
	v_customernumber integer	v_creditlimit integer
1	103	227600

14.2 Prozeduren in PL/pgSQL

Eine Prozedur ist ein benannter PL/SQL Block, der eine bestimmte Aktion ausführt und mit dem sich SQL Abfragen und Logiken automatisieren lassen. Im Gegensatz zu Funktionen gibt eine Prozedur kein Ergebnis zurück. Prozeduren können Variablen speichern und SQL-Abfragen prozedural abarbeiten.

Der Aufbau einer Prozedur entspricht dem Block-Aufbau.

CREATE[OR REPLACE] PROCEDURE

Name [(Parameter_der_Prozedur)]

DECLARE

[lokale Variablen]

BEGIN

Execution;

[EXCEPTION]

END;

\$\$

LANGUAGE

PLPGSQL;

Beispiel:

In unserem Datenmodell *classicmodels* existiert die Tabelle *Orders*, in der Daten der Bestellungen gespeichert werden.

Data Output

Messages

Notifications

	ordernumber [PK] integer	orderdate character varying	requireddate character varying	shippeddate character varying	status text	comments text
1	10100	2003-01-06	2003-01-13	2003-01-10	Shipped	null
2	10101	2003-01-09	2003-01-18	2003-01-11	Shipped	Check on availability.

Wir möchten eine gespeicherte Prozedur *order_delayed* schreiben, die folgendes tut: Sobald das aktuelle Datum größer als das *Requireddate* ist und außerdem das *Shippeddate* noch leer ist, soll der Status der Bestellung automatisch auf *Delayed* gestellt werden, damit diese Bestellung höher priorisiert werden kann.

```
create or replace procedure classicmodels.order_delayed
() as $$
declare
v_requireddate date;
v_shippeddate date;
begin
select orders.requireddate, orders.shippeddate
into v_requireddate, v_shippeddate
from
classicmodels.orders;
update
classicmodels.orders set
Status = 'Delayed'
where v_requireddate < current_date
and v_shippeddate is NULL;
end;
$$ language plpgsql;
```

Aufruf einer Prozedur:

Der Aufruf einer Prozedur erfolgt über ein CALL-Statement. Im Beispiel lautet dieses Statement:

CALL classicmodels.order_delayed ()

Ergebnis: Alle Bestellungen, die verspätet sind, erhalten automatisiert den Status *Delayed*.

Data Output Messages Notifications							
	ordernumber [PK] integer	orderdate character varying	requireddate character varying	shippeddate character varying	status text	comments text	customernumber integer
1	10165	2003-10-22	2003-10-31	null	Delayed	This order ...	148
2	10167	2003-10-23	2003-10-30	null	Delayed	Customer ...	448
3	10248	2004-05-07	2004-05-14	null	Delayed	Order was ...	131
4	10260	2004-06-16	2004-06-22	null	Delayed	Customer ...	357
5	10262	2004-06-24	2004-07-01	null	Delayed	This custo...	141
6	10334	2004-11-19	2004-11-28	null	Delayed	The outsta...	144
7	10401	2005-04-03	2005-04-14	null	Delayed	Customer ...	328
8	10407	2005-04-22	2005-05-04	null	Delayed	Customer ...	450
9	10414	2005-05-06	2005-05-13	null	Delayed	Customer ...	362
10	10420	2005-05-29	2005-06-07	null	Delayed	null	282
11	10421	2005-05-29	2005-06-06	null	Delayed	Custom sh...	124
12	10422	2005-05-30	2005-06-11	null	Delayed	null	157
13	10423	2005-05-30	2005-06-05	null	Delayed	null	314
14	10424	2005-05-31	2005-06-08	null	Delayed	null	141
15	10425	2005-05-31	2005-06-07	null	Delayed	null	119

14.3 Trigger in PL/pgSQL

Ein Trigger ist in einem Datenbankmanagementsystem ein Ereignis, das einen bestimmten Prozess auslöst. Trigger werden als Reaktion auf einen bestimmten Input der Benutzer implementiert und tragen zur Erhaltung der Datenkonsistenz in einem DBMS bei. Mithilfe von Triggern werden Regeln beschrieben, die ähnlich wie Constraints die Datenbank schützen sollen, jedoch noch mächtiger sind. Typische Anwendungsfälle für Trigger sind das Einfügen von Werten in eine bestimmte Tabelle (insert into), das Verändern von Datensätzen in einer Tabelle für bestimmte Einträge (update) oder auch das Löschen von Datensätzen aus einer Tabelle (delete). Zeitpunkte der Ausführung können Before, After und Instead-of Trigger sein.

In PL/pgSQL werden Trigger als spezielle Funktion definiert. Dabei werden Trigger in zwei Schritten definiert:

- a) Definition einer Funktion
- b) Definition eines Triggers, der die Funktion ausführt

Beispiel Before-Trigger:

Ein Before-Trigger validiert eine Anweisung, bevor die Datenbank durch einen Befehl angepasst wird. Für die Tabelle *Customers* im Schema *Classicmodels* sollen Informationen im Terminal angezeigt werden, sobald bestimmte Operationen auf die Tabelle ausgeführt werden. Der Trigger *customers_exception* soll greifen, sobald versucht wird die Tabelle *Customers* zu ändern mit einem

- a) Insert-Statement
- b) Update-Statement
- c) Delete-Statement

Es sollen dabei folgende Texte – je nach Fall – ausgegeben werden. Dabei sollen für die Fälle a) und b) Infos ausgegeben werden, wohingegen für den Fall c) eine Exception ausgelöst werden und ein ERROR zurückgegeben werden soll:

- a) 'Sie haben erfolgreich eingefügt'
- b) 'Sie haben erfolgreich aktualisiert'
- c) 'Sie dürfen Datensätze nicht löschen'

Wenn eine andere Operation ausgeführt wird, soll nichts passieren, der Trigger also nicht greifen. Diese Fälle können wir daher ignorieren (z.B. SELECT).

```
CREATE OR REPLACE FUNCTION classicmodels.customers_exception  
( ) RETURNS trigger AS  
$$  
BEGIN  
CASE TG_OP  
WHEN 'INSERT' THEN  
raise info 'Sie haben erfolgreich eingefügt';  
WHEN 'UPDATE' THEN  
raise info 'Sie haben erfolgreich aktualisiert';  
WHEN 'DELETE' THEN  
raise EXCEPTION 'Sie dürfen Datensätze nicht löschen';  
END CASE;  
RETURN NEW;  
END  
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE TRIGGER customers_exception  
BEFORE INSERT OR UPDATE OR DELETE  
ON classicmodels.customers  
FOR EACH ROW EXECUTE FUNCTION customers_exception ();
```

Ergebnis: Abhängig von der ausgeführten Aktion wird eine der Benachrichtigungen in der Ausgabe angezeigt. *INSERT* und *UPDATE* werden erfolgreich ausgeführt und jeweils eine Info ausgegeben, ein *DELETE* wird über die *EXCEPTION* abgefangen und erzeugt einen Fehler:

```
36 delete from classicmodels.customers  
37 where customernumber = 119  
38
```

Data Output Messages Notifications

ERROR: Sie dürfen Datensätze nicht löschen
CONTEXT: PL/pgSQL function classicmodels.customers_exception() line 9 at RAISE
SQL state: P0001

Beispiel After-Trigger:

Ein After-Trigger führt erst eine Anweisung aus und speichert die Daten in der Datenbank. Im Anschluss wird eine Funktion oder Prozedur ausgelöst, die ein Ergebnis in der Datenbank bewirkt. In den beiden Tabellen *Orders* und *Orderdetails* im Schema *Classicmodels* werden jeweils die *Ordernumbers* gespeichert. Wir nehmen an, dass sich *Ordernumbers* verändern können. Damit wir aber nicht immer beide Tabellen anpassen müssen, wenn sich die *Ordernumber* ändert, möchten wir einen After-Trigger schreiben, der folgendes tut:

Sobald eine *Ordernumber* in der Tabelle *Orders* aktualisiert wird, sollen sich auch automatisiert alle Einträge in der Tabelle *Orderdetails* für diese *Ordernumber* ändern.

```

create or replace function classicmodels.orders_after ()
returns trigger as
$$
begin
update classicmodels.orderdetails
set ordernumber = new.ordernumber
where ordernumber = old.ordernumber;
return new;
end;
$$
language plpgsql;
  
```

```

create or replace trigger orders_update
after UPDATE
on classicmodels.orders
for each row
execute function orders_after ();
  
```

Die Tabelle *Orders* vor der Veränderung:

Data Output Messages Notifications							
	ordernumber [PK] integer	orderdate character varying	requireddate character varying	shippeddate character varying	status text	comments text	customernumber integer
1	10100	2003-01-06	2003-01-13	2003-01-10	Delayed	null	363

Die Tabelle *Orderdetails* vor der Veränderung:

Data Output Messages Notifications					
	ordernumber [PK] integer	productcode [PK] text	quantityordered integer	priceeach double precision	orderlinenumber text
1	10100	S18_1749	30	136	3
2	10100	S18_2248	50	55.09	2
3	10100	S18_4409	22	75.46	4
4	10100	S24_3969	49	35.29	1

UPDATE der *Orders* Tabelle:

```

update classicmodels.orders
set ordernumber = 101005
where ordernumber = 10100
  
```

Ergebnis:

Die Tabelle *Orders* nach der Veränderung:

Data Output Messages Notifications						
	ordernumber [PK] integer	orderdate character varying	requireddate character varying	shippeddate character varying	status text	comments text
1	101005	2003-01-06	2003-01-13	2003-01-10	Delayed	null

Die Tabelle *Orderdetails* nach der Veränderung:

Data Output Messages Notifications					
	ordernumber [PK] integer	productcode [PK] text	quantityordered integer	priceeach double precision	orderlinenumber text
1	101005	S18_1749	30	136	3
2	101005	S18_2248	50	55.09	2
3	101005	S18_4409	22	75.46	4
4	101005	S24_3969	49	35.29	1